

PySpark + SparkSQL Practice Notebook

Gold Layer Reports — Customers & Products | Medallion Architecture Data Warehouse

PySpark

SparkSQL

CTEs

Window Functions

Beginner-friendly

PySpark + SparkSQL Practice Notebook

Gold Layer Reports — Customers & Products

This notebook is a **beginner-friendly PySpark practice** built on top of two gold-layer reports from a medallion architecture data warehouse:

- `gold_report_customers.csv` — one row per customer, with pre-aggregated KPIs (orders, total sales, recency, lifespan, segment, etc.)
- `gold_report_products.csv` — one row per product, with pre-aggregated KPIs (orders, total sales, category/subcategory, product segment, etc.)

Every code cell is commented **line by line** so you can read it like a tutorial. The notebook covers:

- . Starting a Spark session
- . Loading CSVs into DataFrames
- . Exploring schema & data
- . Core DataFrame operations (`select` , `filter` , `groupBy` , `withColumn` , `orderBy`)
- . SparkSQL (temp views, plain SELECTs)
- . SparkSQL **CTEs** (`WITH ... AS (...)`)
- . **Window functions** — both in SparkSQL and in the DataFrame API
- . Same practice repeated on the products table
- . Stopping the Spark session cleanly

. Install & start Spark

If PySpark isn't installed yet, uncomment the pip line below and run it once.

In [1]:

```
# %pip install pyspark # uncomment this line if pyspark is not installed yet

# Import SparkSession, the entry point for using DataFrame & SQL APIs in Spark
from pyspark.sql import SparkSession

# Import common SQL functions under the alias "F" (col, sum, avg, rank, etc.)
from pyspark.sql import functions as F

# Import the Window class, needed to define window specifications later
from pyspark.sql.window import Window

# Build (or reuse) a local SparkSession
# .master("local[*]") -> run Spark locally, using all available CPU cores
# .appName(...) -> a friendly name shown in the Spark UI / logs
# .getOrCreate() -> create a new session, or return the existing one
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("PySpark_SparkSQL_Practice")
    .getOrCreate()
)

# Reduce noisy log output so only warnings and errors are printed
spark.sparkContext.setLogLevel("WARN")

# Print the Spark version to confirm everything started correctly
print("Spark version:", spark.version)
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
26/07/19 15:49:23 WARN Utils: Your hostname, vm, resolves to a loopback
address: 127.0.0.1; using 192.0.2.2 instead (on interface eth0)
26/07/19 15:49:23 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
/usr/local/lib/python3.12/dist-packages/pyspark/testing/utlis.py:127:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some
features may not work correctly. It is recommended to use pandas < 3.0.0
now.
    require_minimum_pandas_version()
26/07/19 15:49:24 WARN NativeCodeLoader: Unable to load native-hadoop lib
for your platform... using builtin-java classes where applicable
Spark version: 4.2.0
```

. Load the gold-layer CSV files into DataFrames

In [2]:

```
# File paths for the two gold report CSVs (already uploaded to the working folder)
customers_path = "gold_report_customers.csv"
products_path = "gold_report_products.csv"

# Read the customers CSV into a Spark DataFrame
# header=True -> first row of the file contains column names
# inferSchema=True -> Spark scans the data and guesses each column's data type
df_customers = spark.read.csv(customers_path, header=True, inferSchema=True)

# Read the products CSV into a Spark DataFrame the same way
df_products = spark.read.csv(products_path, header=True, inferSchema=True)

# Confirm both DataFrames loaded by printing their row counts
print("Customers rows:", df_customers.count())
print("Products rows:", df_products.count())
```

[Stage 1:>
1]

(0 +

Customers rows: 18482
Products rows: 130

. Explore the schema & sample data (customers

In [3]:

```
# printSchema() shows every column name and the data type Spark inferred for it
df_customers.printSchema()
```

```
root
|-- customer_key: integer (nullable = true)
|-- customer_number: string (nullable = true)
|-- customer_name: string (nullable = true)
|-- age: integer (nullable = true)
|-- age_group: string (nullable = true)
|-- customer_segment: string (nullable = true)
|-- last_order_date: date (nullable = true)
|-- recency: integer (nullable = true)
|-- total_orders: integer (nullable = true)
|-- total_sales: integer (nullable = true)
|-- total_quantity: integer (nullable = true)
|-- lifespan: integer (nullable = true)
|-- avg_order_value: integer (nullable = true)
|-- avg_monthly_spend: integer (nullable = true)
```

```
In [4]: # show(5) prints the first 5 rows in a readable table format
# truncate=False -> don't cut off long text values with "..."
df_customers.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|customer_key|customer_number|customer_name  |age|age_group  |
customer_segment|last_order_date|recency|total_orders|total_sales|
total_quantity|lifespan|avg_order_value|avg_monthly_spend|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|1          |AW00011000    |Jon Yang      |54 |50 and above|
VIP        |2013-05-03    |140  |3      |8249      |
8          |8            |2749          |294          |
|2          |AW00011001    |Eugene Huang  |49 |40-49      |
VIP        |2013-12-10    |133  |3      |6384      |
11         |10           |2128          |182          |
|3          |AW00011002    |Ruben Torres  |54 |50 and above|
VIP        |2013-02-23    |143  |3      |8114      |
4          |4            |2704          |324          |
|4          |AW00011003    |Christy Zhu   |52 |50 and above|
VIP        |2013-05-10    |140  |3      |8139      |
9          |9            |2713          |280          |
|5          |AW00011004    |Elizabeth Johnson|46 |40-49      |
VIP        |2013-05-01    |140  |3      |8196      |
6          |6            |2732          |292          |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
```

only showing top 5 rows

```
In [5]: # columns is a plain Python list of column names, useful for quick checks
print(df_customers.columns)
```

```
['customer_key', 'customer_number', 'customer_name', 'age', 'age_group',
'customer_segment', 'last_order_date', 'recency', 'total_orders',
'total_sales', 'total_quantity', 'lifespan', 'avg_order_value',
'avg_monthly_spend']
```

. Core DataFrame operations — select, filter, orderBy

In [6]:

```
# select() keeps only the columns you list, in the order you list them
df_customers.select("customer_name", "customer_segment", "total_sales").show(5)
```

```
+-----+-----+-----+
| customer_name|customer_segment|total_sales|
+-----+-----+-----+
|      Jon Yang|          VIP|      8249|
|    Eugene Huang|          VIP|      6384|
|    Ruben Torres|          VIP|      8114|
|    Christy Zhu|          VIP|      8139|
|Elizabeth Johnson|          VIP|      8196|
+-----+-----+-----+
```

only showing top 5 rows

In [7]:

```
# filter() (alias: where()) keeps only rows that match a condition
# Here: keep only customers in the "VIP" segment
vip_customers = df_customers.filter(F.col("customer_segment") == "VIP")

# Count how many VIP customers exist
print("VIP customers:", vip_customers.count())

# Preview a few of them
vip_customers.select("customer_name", "total_sales", "total_orders").show(5)
```

VIP customers: 1653

```
+-----+-----+-----+
| customer_name|total_sales|total_orders|
+-----+-----+-----+
|      Jon Yang|      8249|          3|
|    Eugene Huang|      6384|          3|
|    Ruben Torres|      8114|          3|
|    Christy Zhu|      8139|          3|
|Elizabeth Johnson|      8196|          3|
+-----+-----+-----+
```

only showing top 5 rows

In [8]:

```
# orderBy() sorts rows by one or more columns
# ascending=False -> sort from highest to lowest (descending)
top_spenders = df_customers.orderBy(F.col("total_sales").desc())

# Show the top 10 highest-spending customers
top_spenders.select("customer_name", "customer_segment", "total_sales").show(10)
```

```
+-----+-----+-----+
| customer_name|customer_segment|total_sales|
+-----+-----+-----+
|Kaitlyn Henderson|VIP|13294|
|Nichole Nara|VIP|13294|
|Margaret He|VIP|13268|
|Randall Dominguez|VIP|13265|
|Adriana Gonzalez|VIP|13242|
|Rosa Hu|VIP|13215|
|Brandi Gill|VIP|13195|
|Brad She|VIP|13172|
|Francisco Sara|VIP|13164|
|Maurice Shan|New|12914|
+-----+-----+-----+
only showing top 10 rows
```

. withColumn — creating derived (calculated) columns

In [9]:

```
# withColumn(new_name, expression) adds a new column, or replaces one if the name already exist
# Here we build a "sales_per_order" metric: total_sales divided by total_orders
df_customers_enriched = df_customers.withColumn(
    "sales_per_order",
    F.round(F.col("total_sales") / F.col("total_orders"), 2) # round to 2 decimals
)

# Show the new column next to the ones it was built from
df_customers_enriched.select(
    "customer_name", "total_sales", "total_orders", "sales_per_order"
).show(5)
```

```
+-----+-----+-----+-----+
| customer_name|total_sales|total_orders|sales_per_order|
+-----+-----+-----+-----+
|Jon Yang|8249|3|2749.67|
|Eugene Huang|6384|3|2128.0|
|Ruben Torres|8114|3|2704.67|
|Christy Zhu|8139|3|2713.0|
|Elizabeth Johnson|8196|3|2732.0|
+-----+-----+-----+-----+
only showing top 5 rows
```

. groupBy + aggregations

In [10]:

```
# groupBy() buckets rows by the value(s) of one or more columns
# agg() then computes summary statistics for each bucket
segment_summary = (
    df_customers.groupBy("customer_segment")          # bucket customers by their segment
    .agg(
        F.count("*").alias("num_customers"),          # how many customers in this segment
        F.sum("total_sales").alias("segment_total_sales"), # sum of all sales in this segment
        F.round(F.avg("total_sales"), 2).alias("avg_sales") # average sales per customer, rounded
    )
    .orderBy(F.col("segment_total_sales").desc())      # sort segments by total sales, highest first
)

# Display the aggregated summary
segment_summary.show()
```

[Stage 18:> 1]

(0 +

```
+-----+-----+-----+-----+
|customer_segment|num_customers|segment_total_sales|avg_sales|
+-----+-----+-----+-----+
|          New|          14629|          11086797|       757.86|
|          VIP|          1653|          10760470|      6509.66|
|        Regular|          2200|          7503991|      3410.91|
+-----+-----+-----+-----+
```

In [11]:

```
# Same idea, but grouped by age_group instead of segment
age_group_summary = (
    df_customers.groupBy("age_group")
    .agg(
        F.count("*").alias("num_customers"),          # customers per age group
        F.round(F.avg("total_sales"), 2).alias("avg_sales") # their average total sales
    )
    .orderBy("age_group")                             # sort alphabetically/naturally by age_group
)

age_group_summary.show()
```

```
+-----+-----+-----+
| age_group|num_customers|avg_sales|
+-----+-----+-----+
|    30-39|          196|   1428.15|
|    40-49|         6103|   1567.44|
|50 and above|       12183|   1601.02|
+-----+-----+-----+
```

. SparkSQL — running plain SQL against a DataFrame

`createOrReplaceTempView` registers a DataFrame as a table name that `spark.sql` query with regular SQL.

In [12]:

```
# Register df_customers as a temporary SQL table named "customers"
df_customers.createOrReplaceTempView("customers")

# Register df_products as a temporary SQL table named "products"
df_products.createOrReplaceTempView("products")

# Now we can write plain SQL against these "tables"
result = spark.sql("""
    SELECT customer_segment,          -- the column we group by
           COUNT(*) AS num_customers, -- how many rows in each group
           SUM(total_sales) AS total_sales -- sum of sales in each group
    FROM customers                    -- the temp view we registered above
    GROUP BY customer_segment        -- one output row per segment
    ORDER BY total_sales DESC        -- highest-spending segment first
""")

# spark.sql() returns a normal DataFrame, so we can call .show() on it
result.show()
```

```
+-----+-----+-----+
|customer_segment|num_customers|total_sales|
+-----+-----+-----+
|          New|         14629|   11086797|
|          VIP|         1653|   10760470|
|        Regular|         2200|    7503991|
+-----+-----+-----+
```

. SparkSQL with a CTE (Common Table Expression)

A CTE is a temporary named result set defined with `WITH name AS (...)`, used to break complex query into readable steps — very similar to what you already practiced in T-SQL.

In [13]:

```
# CTE example: first calculate per-segment totals, then filter on that result
cte_query = spark.sql("""
    WITH segment_totals AS (
        -- Step 1: pre-aggregate sales & order counts per customer segment
        SELECT customer_segment,
               SUM(total_sales) AS segment_sales,
               SUM(total_orders) AS segment_orders
        FROM customers
        GROUP BY customer_segment
    )
    -- Step 2: use the CTE result like a normal table
    SELECT customer_segment,
           segment_sales,
           segment_orders,
           ROUND(segment_sales / segment_orders, 2) AS avg_value_per_order -- derived metric
    FROM segment_totals
    ORDER BY segment_sales DESC
""")

cte_query.show()
```

customer_segment	segment_sales	segment_orders	avg_value_per_order
New	11086797	18788	590.1
VIP	10760470	4213	2554.11
Regular	7503991	4656	1611.68

. Window functions in SparkSQL

Window functions calculate a value *across a set of related rows* (the "window") without coll them into one row, unlike `GROUP BY`. Great for ranking within a group.

In [14]:

```
# RANK customers by total_sales, restarting the ranking for each customer_segment
window_query = spark.sql("""
    SELECT customer_name,
           customer_segment,
           total_sales,
           RANK() OVER (
               PARTITION BY customer_segment -- restart ranking for each segment
               ORDER BY total_sales DESC    -- highest sales gets rank 1
           ) AS rank_in_segment
    FROM customers
""")

# Keep only the top 3 customers within each segment
top3_per_segment = window_query.filter(F.col("rank_in_segment") <= 3)

top3_per_segment.orderBy("customer_segment", "rank_in_segment").show(30)
```

customer_name	customer_segment	total_sales	rank_in_segment
Maurice Shan	New	12914	1
Janet Munoz	New	12488	2
Ricky Navarro	New	7292	3
Joe Sanchez	Regular	4987	1
Erick Sai	Regular	4985	2
Erica Hu	Regular	4978	3
Kaitlyn Henderson	VIP	13294	1
Nichole Nara	VIP	13294	1
Margaret He	VIP	13268	3

. The same window function using the DataFrame API (no SQL text)

In [15]:

```
# Define a window specification: partition by segment, order by total_sales descending
segment_window = Window.partitionBy("customer_segment").orderBy(F.col("total_sales").desc())

# Apply the window function with withColumn(), just like a normal derived column
df_ranked = df_customers.withColumn(
    "rank_in_segment",
    F.rank().over(segment_window) # .over() attaches the window spec to the function
)

# Filter to the top 3 per segment, same result as the SQL version above
df_ranked.filter(F.col("rank_in_segment") <= 3) \
    .select("customer_name", "customer_segment", "total_sales", "rank_in_segment") \
    .orderBy("customer_segment", "rank_in_segment") \
    .show(30)
```

customer_name	customer_segment	total_sales	rank_in_segment
Maurice Shan	New	12914	1
Janet Munoz	New	12488	2
Ricky Navarro	New	7292	3
Joe Sanchez	Regular	4987	1
Erick Sai	Regular	4985	2
Erica Hu	Regular	4978	3
Kaitlyn Henderson	VIP	13294	1
Nichole Nara	VIP	13294	1
Margaret He	VIP	13268	3

. Switching to the products dataset

In [16]:

```
# printSchema() again, this time for the products DataFrame  
df_products.printSchema()
```

```
root  
|-- product_key: integer (nullable = true)  
|-- product_name: string (nullable = true)  
|-- category: string (nullable = true)  
|-- subcategory: string (nullable = true)  
|-- cost: integer (nullable = true)  
|-- last_sale_date: date (nullable = true)  
|-- recency_in_months: integer (nullable = true)  
|-- product_segment: string (nullable = true)  
|-- lifespan: integer (nullable = true)  
|-- total_orders: integer (nullable = true)  
|-- total_sales: integer (nullable = true)  
|-- total_quantity: integer (nullable = true)  
|-- total_customers: integer (nullable = true)  
|-- avg_selling_price: double (nullable = true)  
|-- avg_order_revenue: integer (nullable = true)  
|-- avg_monthly_revenue: integer (nullable = true)
```

In [17]:

```
# Preview the first 5 product rows
df_products.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+
|product_key|product_name          |category|subcategory  |cost|
last_sale_date|recency_in_months|product_segment|lifespan|total_orders|
total_sales|total_quantity|total_customers|avg_selling_price|
avg_order_revenue|avg_monthly_revenue|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+
|3          |Mountain-100 Black- 38 |Bikes    |Mountain Bikes|1898|
2011-12-27  |157                  |High-Performer |11          |49  |
165375      |49                   |49          |3375.0       |3375
|15034      |                      |
|4          |Mountain-100 Black- 42 |Bikes    |Mountain Bikes|1898|
2011-12-27  |157                  |High-Performer |11          |45  |
151875      |45                   |45          |3375.0       |3375
|13806      |                      |
|5          |Mountain-100 Black- 44 |Bikes    |Mountain Bikes|1898|
2011-12-21  |157                  |High-Performer |11          |60  |
202500      |60                   |60          |3375.0       |3375
|18409      |                      |
|6          |Mountain-100 Black- 48 |Bikes    |Mountain Bikes|1898|
2011-12-26  |157                  |High-Performer |12          |57  |
192375      |57                   |57          |3375.0       |3375
|16031      |                      |
|7          |Mountain-100 Silver- 38 |Bikes    |Mountain Bikes|1912|
2011-12-22  |157                  |High-Performer |12          |58  |
197200      |58                   |58          |3400.0       |3400
|16433      |                      |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+
only showing top 5 rows
```

In [18]:

```
# filter() example: keep only products in the "Bikes" category
bikes = df_products.filter(F.col("category") == "Bikes")

print("Bike products:", bikes.count())
bikes.select("product_name", "subcategory", "total_sales").show(5)
```

Bike products: 88

```
+-----+-----+-----+
|      product_name|  subcategory|total_sales|
+-----+-----+-----+
|Mountain-100 Blac...|Mountain Bikes|    165375|
|Mountain-100 Blac...|Mountain Bikes|    151875|
|Mountain-100 Blac...|Mountain Bikes|    202500|
|Mountain-100 Blac...|Mountain Bikes|    192375|
|Mountain-100 Silv...|Mountain Bikes|    197200|
+-----+-----+-----+
only showing top 5 rows
```

. GroupBy & aggregation on products

In [19]:

```
# Total and average sales per category
category_summary = (
    df_products.groupBy("category")
    .agg(
        F.count("*").alias("num_products"),           # how many products in this category
        F.sum("total_sales").alias("category_total_sales"), # category's combined sales
        F.round(F.avg("total_sales"), 2).alias("avg_product_sales") # average sales per product
    )
    .orderBy(F.col("category_total_sales").desc())    # highest-selling category first
)

category_summary.show()
```

```
+-----+-----+-----+-----+
|  category|num_products|category_total_sales|avg_product_sales|
+-----+-----+-----+-----+
|      Bikes|         88|        28311657|    321723.38|
|Accessories|         22|         699909|     31814.05|
|   Clothing|         20|         339692|     16984.6|
+-----+-----+-----+-----+
```

. SparkSQL CTE on products

In [20]:

```
# CTE example: rank subcategories by total sales within each category
product_cte = spark.sql("""
    WITH subcategory_totals AS (
        -- Step 1: sum sales per category + subcategory combination
        SELECT category,
               subcategory,
               SUM(total_sales) AS subcategory_sales
        FROM products
        GROUP BY category, subcategory
    )
    -- Step 2: keep only subcategories above the overall average subcategory sales
    SELECT *
    FROM subcategory_totals
    WHERE subcategory_sales > (SELECT AVG(subcategory_sales) FROM subcategory_totals)
    ORDER BY subcategory_sales DESC
""")

product_cte.show(truncate=False)
```

```
+-----+-----+-----+
|category|subcategory|subcategory_sales|
+-----+-----+-----+
|Bikes   |Road Bikes  |14519438          |
|Bikes   |Mountain Bikes|9947639          |
|Bikes   |Touring Bikes|3844580           |
+-----+-----+-----+
```

. Window function on products — rank inside category

In [21]:

```
# Rank each product by total_sales, restarting at 1 for every category
product_window = spark.sql("""
    SELECT product_name,
           category,
           total_sales,
           DENSE_RANK() OVER (
               PARTITION BY category    -- restart ranking per category
               ORDER BY total_sales DESC -- best-selling product gets rank 1
           ) AS rank_in_category
    FROM products
""")

# Keep only the #1 best-seller in each category
best_in_category = product_window.filter(F.col("rank_in_category") == 1)

best_in_category.orderBy("category").show(truncate=False)
```

```
+-----+-----+-----+-----+
|product_name          |category   |total_sales|rank_in_category|
+-----+-----+-----+-----+
|Sport-100 Helmet- Red  |Accessories|78050      |1               |
|Mountain-200 Black- 46 |Bikes      |1373454    |1               |
|Women's Mountain Shorts- L|Clothing   |25410      |1               |
+-----+-----+-----+-----+
```

. Wrap-up — stop the Spark session

In [22]:

```
# Always stop the SparkSession when you're done, to free up local resources
spark.stop()

print("Spark session stopped. Practice complete!")
```

Spark session stopped. Practice complete!

What you practiced

- Creating a `SparkSession` and reading CSVs into DataFrames
- `select`, `filter`, `orderBy`, `withColumn`, `groupBy` + `agg`
- Registering DataFrames as SQL temp views with `createOrReplaceTempView`
- Writing SparkSQL queries with `spark.sql()`
- **CTEs** (`WITH ... AS (...)`) to break queries into readable steps
- **Window functions** (`RANK`, `DENSE_RANK`) both in SQL and in the DataFrame API

Suggested next steps

- Try `ROW_NUMBER()` vs `RANK()` vs `DENSE_RANK()` and compare the differences
- Add a running total with `SUM(...)` `OVER (ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
- Try `LAG()` / `LEAD()` to compare a customer's sales to the "next" customer in a row